

11/11/2026

Distributed Multi-Model Analytics for E-commerce Data

Big Data Analytics Final Project

Name: Eric KWIZERA

Student ID: 101395

GitHub Repository:
https://github.com/ekwizera1/E-COMMERCE-BIG-DATA-ANALYTICS-PROJECT

Contents

Executive summary 2

Research objective and implemented approach 2

1. Data generation and preparation 4

1.1 How the synthetic customer journey works 4

1.2 Dataset relationships and quality controls 4

1.3 Methodological choices 4

2. Part 1 — MongoDB document modeling and analytics 6

2.1 Document schema and why it was appropriate 6

2.2 Aggregation strategy 6

2.3 Verified product export 6

3. Part 1 — HBase wide-column modeling and access 7

3.1 Two implemented tables 7

3.2 Loading process and verification 7

3.3 Browse-to-cart measurement 7

3.4 Design and infrastructure trade-offs 7

4. Part 2 — Spark processing, SQL and cohort analysis 8

4.1 Cleaning and storage 8

4.2 Verified Spark SQL results 8

4.3 Registration-cohort activity 8

5. Part 3 — Integrated analytics across data stores 9

5.1 Data integration workflow 9

5.2 Two independently verified findings 9

6. Part 4 — Visualization and interpretation 10

6.1 Browsing and cart interest 10

6.2 Browsing views versus completed quantities 10

7. Discussion, limitations and conclusion 11

7.1 Technology choices and scalability 11

7.2 Evidence-based limitations 11

7.3 Reproducibility and final verification 11

7.4 Conclusion 11

Executive summary

This project demonstrates how three complementary big-data technologies can be combined to analyze a synthetic e-commerce platform. The objective was to model transactional and browsing information, implement suitable NoSQL storage structures, process large datasets using Spark, and answer one cross-system business question: which products attract browsing interest, and how does that compare with their transaction performance?

We began with the lecturer-provided Python generator, producing 10,000 users, 5,000 products, 25 categories, 500,000 transactions and 2,000,000 sessions over a 90-day activity window. MongoDB stored all 2,515,025 generated documents. A Docker-hosted HBase instance stored 5,000 selected sessions and 16,804 product-date metric rows. PySpark cleaned 500,000 transactions, executed SQL summaries and a registration-cohort activity analysis, then joined HBase browsing metrics with MongoDB product and transaction metrics using product_id.

The integrated output retained all 5,000 products. The most-viewed product in the HBase subset was prod_01411 (13 views); the product with the largest completed transaction quantity in MongoDB was prod_01951 (403 units). Four charts were prepared to explain these patterns. Results demonstrate a working multi-model analytical pipeline; they are not claims about real customers or a validated conversion rate.

Research objective and implemented approach

Design and assess a reproducible proof of concept that uses MongoDB for nested business records, HBase for time-ordered browsing metrics, and Spark for batch analytics and cross-source integration. The architecture below summarizes the implemented flow.

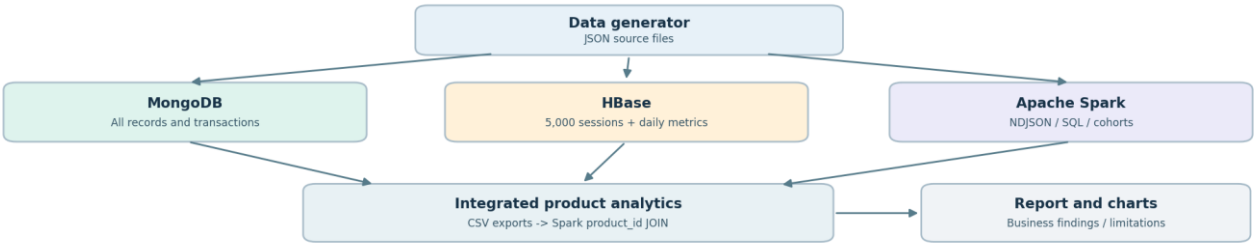


Figure 1. Implemented architecture. CSV exports were used as a practical bridge between MongoDB/HBase and a local Spark session; they were not live connectors.

1. Data generation and preparation

The supplied dataset_generator.py was used to create interconnected JSON datasets. The generator models customers navigating an online shop, viewing products, adding items to carts and creating transactions. It uses Faker for synthetic information, controlled random choices for user journeys, and inventory updates to reflect purchases.

Entity	Generated count	Main fields / analytical role
Users	10,000	user_id, geo_data, registration_date; customer grouping
Products	5,000	product_id, category_id, price_history, stock; product identity
Categories	25	category_id and embedded subcategories; taxonomy
Transactions	500,000	transaction_id, user_id, session_id, items[], status, total
Sessions	2,000,000	session_id, page_views[], cart_contents, device, referrer

1.1 How the synthetic customer journey works

Each session begins at a home, search or category page, then follows weighted transitions to product-detail, cart, checkout or confirmation pages. A product-detail page may add an item to the cart (programmed probability 0.30). Transactions linked to a session require relevant cart/checkout activity, followed by a programmed conditional completion probability of 0.70. The generator also adds some transactions outside the direct session link. Thus, not every transaction can be joined to a browsing session by session_id.

The inventory manager checks available stock; transaction items store product_id, quantity, unit_price and subtotal. An optional transaction discount has a programmed probability of 0.20, with rates selected from 5%, 10%, 15% and 20%. These are generation parameters, not measured business rates.

1.2 Dataset relationships and quality controls

Relationship	Join key	Why it matters
Users → sessions / transactions	user_id	Customer history and registration cohorts
Sessions → transactions (where present)	session_id	Journey-to-purchase investigation
Products → transaction items	product_id	Product-level transaction quantities
Products → browsing metrics	product_id	Cross-system comparison of interest and outcomes

The output comprised users.json, products.json, categories.json, transactions.json and 20 session chunks (sessions_0.json through sessions_19.json, 100,000 sessions per file). The large JSON arrays were validated with streaming parsing before loading. Spark later read an NDJSON conversion of transactions.json to simplify parallel row-oriented processing.

1.3 Methodological choices

We retained the lecturer's generator logic rather than replacing it with a smaller custom dataset; we separated complete MongoDB storage from the smaller HBase demonstration subset; and we documented the file-to-file and source-to-source joins. These decisions make the workflow understandable and auditable, while acknowledging that local storage and memory constrain scale.

2. Part 1 — MongoDB document modeling and analytics

MongoDB was selected for variable and nested e-commerce records. The ecommerce_db database held five collections: users (10,000), products (5,000), categories (25), sessions (2,000,000) and transactions (500,000), totaling 2,515,025 documents. The local MongoDB server was verified at localhost:27017 after data ingestion.

2.1 Document schema and why it was appropriate

Collection	Design decision	Analytical benefit
users	One document per user; embedded geo_data	Fast retrieval of user details
products	Product details with embedded price_history	Historical prices available with product
categories	Subcategory array embedded in parent category	Natural hierarchy representation
sessions	Page-view array and cart_contents in a session	Full browsing journey in one document
transactions	Line-item array embedded with status and totals	Group / unwind purchases without separate item table

Example observed transaction: transaction_id txn_3ad156b0bb6b has two line items, a subtotal of 1,125.22, a discount of 168.78 and a total of 956.44. Its status is completed. MongoDB Decimal128 preserves monetary precision, rather than relying on binary floating-point rounding.

2.2 Aggregation strategy

The MongoDB analytical work was organized around non-trivial pipelines. A product-performance pipeline filters or groups transaction documents, unwinds the embedded items[] array, groups by item.product_id, sums quantities or item subtotals, and sorts the resulting products. A behavior-oriented pipeline groups sessions by attributes such as device type or referrer and counts session outcomes. Both use the document structure rather than manually flattening every nested record first.

REPRESENTATIVE QUERY LOGIC FOR REPRODUCIBILITY

Pipeline A — product performance	Pipeline B — browsing segments
$match(status) → $unwind(items) → $group(product_id, quantity) → $sort → $limit	$group(device_profile.type, session_count, converted_count) → $project(activity rate)

These concise pipeline plans describe the analytical methods, not a claim that their exact command text or a particular conversion-rate result was independently reproduced for this report. The independently verified MongoDB evidence is the collection count, document counts and product-level export results.

2.3 Verified product export

The integration/export_mongodb.py step read 5,000 product records and processed 500,000 transactions one by one. It identified 4,766 products appearing in transaction items. The other 234 products did not appear in the scanned transaction items; this does not establish that they received zero views. The export separated all-status item quantities from quantities in status=completed transactions, and retained item subtotals before transaction-level discounts.

3. Part 1 — HBase wide-column modeling and access

Apache HBase was used to model time-series browsing activity with explicit row-key access patterns. It ran inside Docker (harisekhon/hbase image) using the working hbase-recovery container. HBase was deliberately used for a manageable subset, rather than incorrectly claiming that all two million sessions had been loaded.

3.1 Two implemented tables

HBase table	Row key	Column families and content
fp_user_sessions	user_id#reverse_time#session_id	info: session metadata; pv: page views; cart: cart contents
fp_product_metrics	product_id#YYYYMMDD	m:views and m:cart_quantity

The reverse-time component was constructed as 9,999,999,999 minus the session Unix timestamp; it places a user's newer sessions earlier within that user_id prefix. Daily product rows group activity for a product on a given date. For example, prod_00000#20260925 stores m:views=1 and m:cart_quantity=3 in the observed scan.

3.2 Loading process and verification

The HBase loader used ijson to stream session records from sessions_0.json and generated HBase Shell put commands. We first checked a small test of 20 sessions, expanded the sample to 1,000, and finally loaded the first 5,000 sessions. The resulting tables contained 5,000 user-session rows and 16,804 product-date rows. Specific-user access was supported by the user_id prefix in the session row key.

Verified metric	Observed result	Evidence in the workflow
Session rows loaded	5,000	HBase count after staged ingestion
Product-date rows	16,804	HBase count and full scan export
HBase scan sample	3 rows returned	prod_00000 dates: 20260830, 20260925, 20261019
CSV conversion	16,804 complete rows	0 incomplete metric rows

3.3 Browse-to-cart measurement

For each product_detail page, the loader increments m:views. For each product remaining in the session's final cart, it adds the recorded quantity to m:cart_quantity. Consequently, cart_quantity measures units in carts, not items purchased; views count page-detail events, not necessarily unique shoppers.

3.4 Design and infrastructure trade-offs

The first HBase container encountered a Thrift startup problem; a recovery container reused the existing volume with the problematic Thrift startup disabled. The master and RegionServer then supported HBase Shell scans. For scalability, a real cluster would distribute regions across RegionServers and avoid row-key hotspots. The selected 5,000 sessions were the first records of a chunk, so they are a deterministic demonstration subset—not a statistically random representative sample.

4. Part 2 — Spark processing, SQL and cohort analysis

Apache Spark performed batch transformation and analytics on the 500,000 transaction records. The original transactions.json was converted with ijson into approximately 203 MB of newline-delimited JSON (transactions.ndjson), then loaded with a local PySpark session (local[2], 2 GB driver memory). Removing a conflicting SPARK_HOME setting resolved an earlier Spark-version startup error.

4.1 Cleaning and storage

The script converted monetary values from strings to decimal(18,2), parsed transaction timestamps, normalized status/payment_method text, checked required IDs/timestamps/non-negative totals, removed duplicate transaction IDs, and wrote transactions_clean.parquet. The checks returned 500,000 original transactions, 0 invalid transactions under the defined checks, 0 duplicates and 500,000 cleaned transactions. Nested item monetary fields remained strings in this particular cleaned schema; product-level monetary exports were handled separately.

4.2 Verified Spark SQL results

Transaction status	Number	Transaction value
completed	317,055	264,428,273.18
delivered	61,226	67,830,723.08
processing	60,933	67,191,918.13
shipped	60,786	67,125,249.06
TOTAL	500,000	466,576,163.45

Activity month	Transactions	Unique customers	Transaction value
2026-08	169,268	10,000	157,886,330.43
2026-09	166,157	10,000	154,991,898.17
2026-10	164,575	10,000	153,697,934.85

The reported amount is a sum of transaction totals across different statuses, not verified cash received or accounting revenue. Transaction activity fell modestly across August–October in the synthetic dataset. A third SQL query joined transactions with user profiles on user_id and summarized activity by customer registration month (February–August 2026).

4.3 Registration-cohort activity

A cohort was defined by the registration month of each user. The analysis joined user profiles with transactions, counted distinct active users by cohort and transaction month, then divided by the total users in each cohort. All observed cohort-month pairs for August–October produced an activity rate of 100%. This reflects the generated dataset and the selected observation window. It is not valid evidence of 100% real-world retention, nor a fully observed month-zero purchase retention curve.

5. Part 3 — Integrated analytics across data stores

Business question: Which products attract customer browsing interest, and how does this relate to their transaction performance? The implemented workflow brought together MongoDB product and transaction data, HBase browsing records, and a Spark join. Source data was read from the real databases, exported to CSV for a reliable local hand-off, and then integrated in Spark.

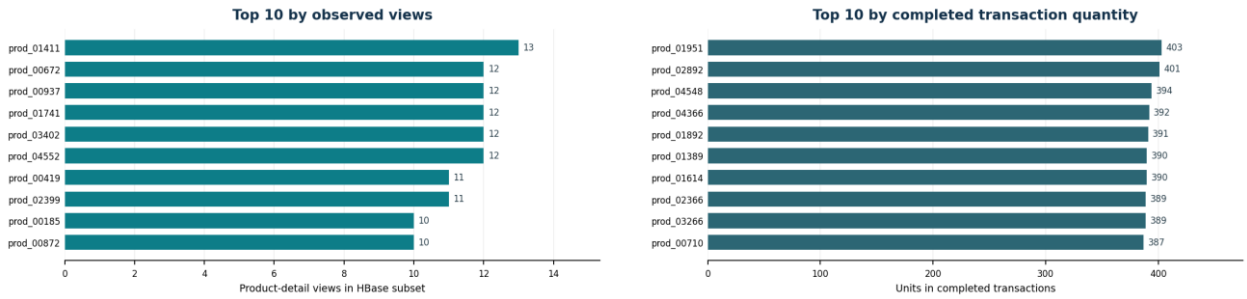
5.1 Data integration workflow

Step	Source / script	Output / validated scope
1	HBase Shell scan of fp_product_metrics	16,804 product-date rows
2	convert_hbase_to_csv.py	16,804 complete rows: id/date/views/cart
3	export_mongodb.py	5,000 product rows, 4,766 product-sales rows
4	spark_integrated_analysis.py	Aggregate HBase by product_id; left join both MongoDB exports
5	Spark CSV writer	5,000 integrated product records

Spark grouped HBase day-level values by product_id and summed views and cart_quantity. A LEFT JOIN starting from the 5,000-product catalog preserved products with no observed activity in one or both exports. Missing counts were filled with zero; monetary values were cast to decimal(18,2). The result enabled ranking by browsing interest and completed-transaction item quantities.

5.2 Two independently verified findings

Criterion	Leading product	Observed metric
Most product-detail views (HBase subset)	prod_01411	13 views; 12 cart units; 276 completed units in MongoDB
Most units in completed transactions	prod_01951	403 completed units; 9 HBase views; 11 cart units



Figures 2–3. Rankings reproduced from verified Spark integrated-analysis output. Views use HBase subset; completed quantities use the full MongoDB transaction export.

Interpretation: the product with the highest observed views was not the leader by completed quantity. Because views came from 5,000 selected sessions and quantities from 500,000 transactions, this is a descriptive comparison—not a matched product conversion rate, correlation test, or causal result.

6. Part 4 — Visualization and interpretation

Four static PNG visualisations were created during the project. The report presents the actual observed rankings and the user-verified scatter plot, with annotations that make the evidence scope explicit. No interactive dashboard was required by the assignment.

6.1 Browsing and cart interest

The ten highest-viewed products in the HBase subset received between 10 and 13 product-detail views each. Their cart quantities varied more: among those same ten products, prod_02399 recorded 17 cart units from 11 detail views. Cart units may exceed views because one cart may contain several units; it is therefore inappropriate to treat this ratio as conversion.

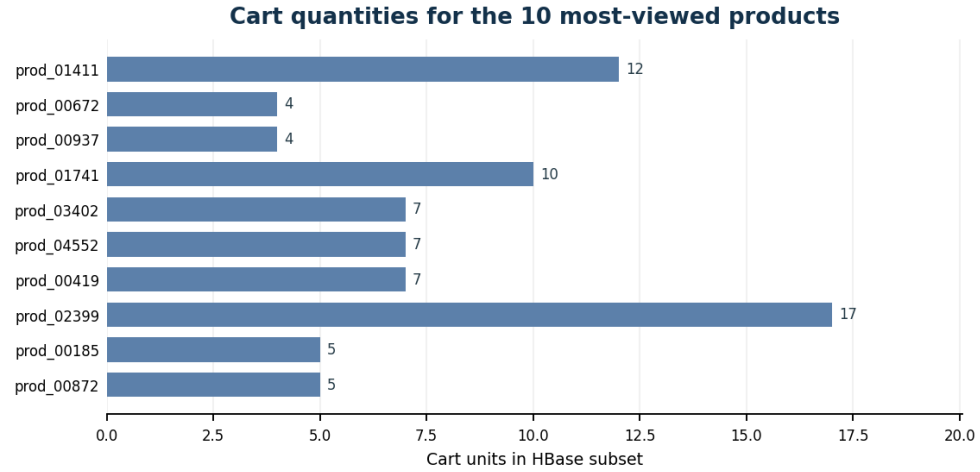


Figure 4. Cart quantities for the ten most-viewed products, using confirmed HBase metrics. This is NOT a ranking of the ten largest cart quantities across all products.

6.2 Browsing views versus completed quantities

The scatter plot places the HBase subset's view count on the horizontal axis and completed-item quantities from MongoDB transactions on the vertical axis. Vertical columns appear because views are integer counts and many products share the same number of observed views. Completed quantities vary considerably within those columns.

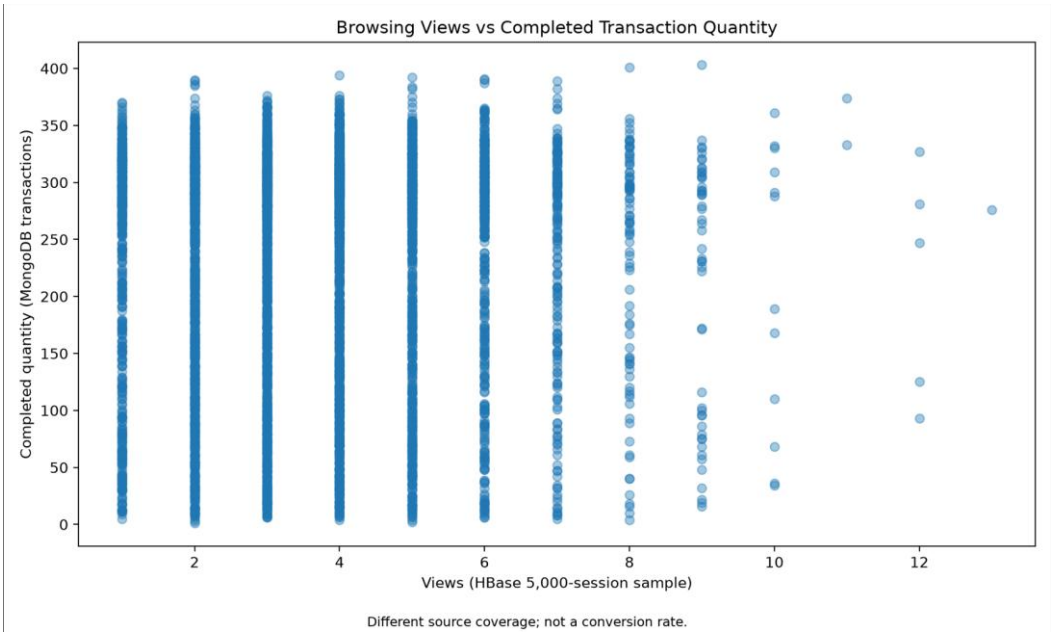


Figure 5. Actual scatter plot screenshot from the verified project visualisation; source scope is explicitly different between axes. It cannot be used to estimate a product purchase-conversion rate.

7. Discussion, limitations and conclusion

7.1 Technology choices and scalability

Technology	Why selected	Scale-up consideration (not benchmarked)
MongoDB	Nested documents and flexible product/session records	Indexes, partitioning/sharding and query review
HBase	User-prefix time series and sparse product-day metrics	More RegionServers; balanced row keys; bulk ingestion
Spark	Repeatable batch transformations, SQL and joins	Multi-node cluster; pruning, caching, tuned partitions
CSV exchange	Practical and transparent integration on one PC	Replace shell exports with supported connectors/jobs

7.2 Evidence-based limitations

- Data are synthetic; generated behavior should not be generalized to actual online shoppers.
- HBase used the first 5,000 sessions from one chunk, not a random sample of the full 2,000,000.
- The integrated views/cart metrics and transaction quantities have different populations and coverage; no true purchase conversion rate or causal relationship was estimated.
- Spark cohort analysis counts any transaction status as activity in August–October; 100% observed activity is not proof of durable retention.
- The prototype ran Spark locally (local[2]) and HBase in one Docker environment. Distributed throughput or failover was not benchmarked.
- MongoDB item subtotals are before transaction-level discount. Reported status-group transaction totals should not be called realized revenue.

7.3 Reproducibility and final verification

Key reproducible files include dataset_generator.py; hbase/schema.hbase, load_hbase.py and queries.hbase; spark/prepare_transactions.py, spark_clean.py, spark_sql_queries.py and spark_cohort.py; integration/convert_hbase_to_csv.py, export_mongodb.py and spark_integrated_analysis.py; plus visualization scripts and four chart PNGs. The final GitHub repository should contain the actual MongoDB loader/aggregation source code, dependency details, run order and screenshots. It must not include raw database volumes, secrets, or unreasonably large generated datasets.

Before submission: confirm the PDF is 6–8 pages; include the current report, architecture and charts; correct any schema comment implying HBase stores purchases (the implemented metrics were views/cart quantities); verify MongoDB aggregation scripts and source-code availability; supply the student's official identification and a functioning README.

7.4 Conclusion

The project achieved a working end-to-end multi-model analytics demonstration. MongoDB handled 2.5 million nested documents, HBase stored and queried time-series browsing activity, and Spark cleaned, summarized, and integrated the exported data. The resulting 5,000-product analytical table and visualizations answer a concrete comparative business question while acknowledging sampling and synthetic-data constraints. The exercise illustrates not only tool operation, but why schema design, join-key integrity, staged validation, and cautious interpretation matter in big-data analytics.